Tabular Reinforcement Learning: *a small agent in a big world*

Ali Momen (s4993128) [1] **Bram Bouma (s2646072)** [1] **Judith Moulijn (s3206025)** [1]

Abstract

This article is about implementing Reinforcement Learning various algorithms on a grid world problem with stochastic factors. The algorithms are run in enough explained iterations to use the previous learnings, to be able to converge to the terminal state in less steps. Every algorithm is also run multiple times to generate more results so as to be more reliably comparable with the other algorithms. The approaches and algorithms included are "**Dynamic Programming (DP)**", "**Exploration Techniques**", "**On-policy versus Off-policy**" and "**Depth of Target**".

1. Introduction

Finding an optimal path in a given situation can be modelled using tabular reinforcement learning (RL). In this regime of RL the state and actions spaces are discrete. Relatively low-dimensional "grid worlds" in this regime allow for both state and action spaces to be stored in their entirety. Here we will cover several methods for finding the optimal path through a certain environment.

In section 2 the theory relevant for the methods used is covered. This will be based on the work in (Plaat, 2022). In section 3 the different experiments applied to find the optimal path through this environment are covered and their results are presented. In section 4 these results are discussed. We end with a reflection in section 5

2. Theory

To be able to find the optimal path through the environment used, this sequential decision problem (SDP) will be modelled as a Markov decision process (MDP). An agent's next action is taken solely based on the information available at a given state. Taking all past experiences into account, becomes complicated. To model an SDP as an MDP the following things are needed: the action space A, the state space S, the transition probability $T(s'|a,s)$, the reward associated with state s: r_s and the discount factor γ. The goal is to find the optimal policy $\pi^\star$ which maximises the expected return, or

$$\pi^\star = \arg\max_{a \in A} Q^\star(s,a) \tag{1}$$

With $Q^\star(s,a)$ the optimal state-action value function. $Q^\pi(s,a)$ is the expected return if a given policy $\pi(a|s)$ is used to decide what action to take after being in state s and taking action a.

3. Experiments

Let's now go over the environment that will be used throughout this report. The environment in which the agent is moving around is a 7x10 grid world. The agent starts at a starting position (0,3). A goal location is set at (7,3) and can be moved around. Reaching the goal state yields a reward of 100. Reaching any other state yields a reward of -1. In columns 3, 4, 5 and 8 there's a 90% change that the agent will be moved upwards one tile, due to a stochastically present wind. In columns 6 and 7 this results in an upwards movement of two tiles with the same probability.

3.1. Dynamic programming

A method of constructing the state-action value function $Q(s,a)$ is by using the Value Iteration algorithm (see Algorithm 1). This algorithm loops over both state and action spaces to iteratively improve the Q table until the state-actions values don't change much any more (then convergence is reached). The output of Algorithm 1 is an optimal (within the constraints of the parameters used) Q table which can be used to help the agent reach the goal. In a given state s, the next action a' is chosen based on the action that maximises the $Q(s,a')$ value. This is called a *greedy* policy. This algorithm requires the transition probability to be known from the environment and is therefore a model-based solution method for the MDP.

The converged optimal value at the start state can be found using the value function (Plaat, 2022):

$$V^\pi(s) = \mathbb{E}\left[\sum_{i=0}^{N} \gamma^i \cdot r_{t+i} | s_t = s\right] \tag{2}$$

[*]Equal contribution [1]Universiteit Leiden, The Netherlands. Correspondence to: Ali Momen <amomen@gmail.com>, Bram Bouma <bram.f.bouma@gmail.com>, Judith Moulijn <j.moulijn@umail.leidenuniv.nl>.

With γ the discount factor, r_t the reward received at timestep t and N the number of steps taken until the goal is reached. This can be used in $V(s_{\text{initial}})$ to get an expected return for the entire episode (from the initial state up to when the goal is reached).

The main hyperparameters used in this experiment are the discount factor γ and the threshold η. Setting γ lower will mean that the Q table is built up by taking only the closest rewards into account (see also Figure 7). Setting γ equal to one, means all future rewards are taken into account equally. Setting η higher means that convergence is reached earlier. If the steps to states other than the terminal state awarded 0 instead of -1, the shorted path could still be found using $\gamma < 1$. By only considering a few states ahead, too much exploring is discouraged: this would yield no reward.

Using $\gamma = 0.99$ and $\eta = 70$. Algorithm 1 converges in 12 iterations. Then applying Equation 2 on the initial state yields $V(s_{\text{initial}}) \approx 70$. The average reward per timestep can be calculated using the reward in the terminal state (R_{terminal}), the fact that each step to a different state yields a reward of -1 and the number of steps taken (N_{steps}). The average reward per timestep is then

$$\bar{r_t} = (100 + (n_{\text{steps}} - 1) \cdot -1) \, / n_{\text{steps}} \tag{3}$$

A single step is subtracted from n_{steps} inside the equation because the final step into the terminal state doesn't yield a reward of -1 but one of 100 as previously stated. On average, using the same values for the hyperparameters γ and η, it takes about 17 steps to reach the terminal state. This means that $\bar{r_t} \approx 4.9$.

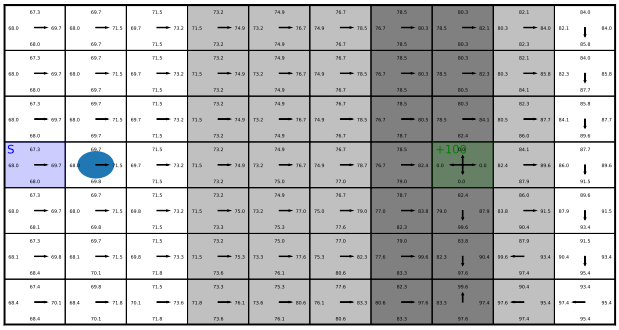

Figure 1. After 12 iterations of value iteration, convergence is reached (when using $\gamma = 0.99$ and $\eta = 70$.). For ealier stages of the algorithm see Figure 5 and Figure 6. The agent will move to the right, end up in the top-right corner due to the stochastic wind, then work its way to the bottom-right and then to the left and up towards the goal.

3.2. Exploration

In this experiment the next action will be chosen either as before (greedy) or by choosing a different action with a small chance. The selection between these two is done using either "**e-greedy**" (see Equation 4) or "**Boltzman (softmax)** (see Equation 5)" methods. The stochasticity is applied through one of these methods. The relative performance between these two methods will be compared by drawing a plot (seeFigure 12).

$$\pi(a \mid s) = \begin{cases} 1.0 - \epsilon \cdot \dfrac{|A| - 1}{|A|}, & \text{if } a = \arg\max_{b \in A} \hat{Q}(s, b) \\ \dfrac{\epsilon}{|A|}, & \text{otherwise} \end{cases} \tag{4}$$

The upper case covers exploitation and the lower case covers exploration which chooses one from the rest of non-greedy actions.

$$\pi(a \mid s) = \frac{e^{\hat{Q}(s,a)/\tau}}{\sum_{b \in A} e^{\hat{Q}(s,b)/\tau}} \tag{5}$$

It also gives a probability to every possible action. In this experiment the Q table is constructed using tabular Q-learning (see Algorithm 2). The hyperparameters in experiments generally include **learning rate** α, **exploration rates** ϵ and τ for e-greedy and softmax methods respectively, **number of timesteps** (budget), γ (**discount factor**), and **evaluation interval**. We try to tune them as far as we can. The hyperparameters and figures and expanded interpretation for the first and last trials are in the appendix section, and only the summary interpretation is done here.

Hyperparameters Optimization

To hasten convergence, the learning rate could be increased and the exploration rates adjusted. Increasing the learning rate and keeping other parameters static, as expected, lead to two results: 1. Faster convergence (Pro) 2. Volatile behaviour of some curves, slow convergence, oscillation, and less satisfactory **Returns** (Con) We reacted to this by decreasing the exploration rates. This resulted in faster convergence, but also had steady and smooth behaviour. The evaluation interval was not changed to keep the plots smoother (for a numerical overview, see also subsubsection .3.1). To see the influence of an additional terminal state with a reward of 5 on the agent's behaviour, three experiments were run: one for every additional location (added at [3,2], [7,4] and [8,4]). The impact is that the mean of returns, convergence speed, and hyperparameter effects change. The location of this additional goal strongly influences its effect. Some locations (closer to the starting position, for example) are more likely to be visited. See Figure 13, Figure 14 and Figure 15 for some visualisations of the impact of this additional goal.

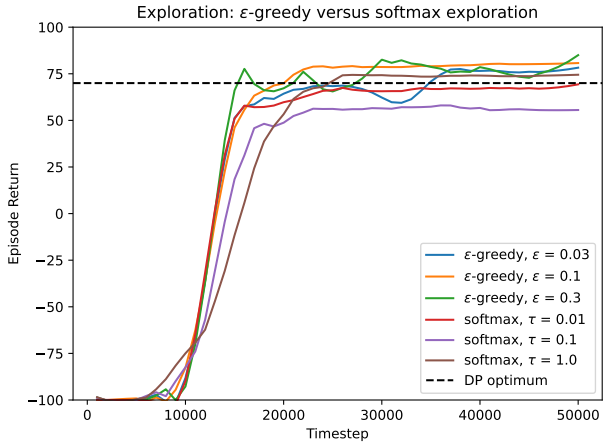

Figure 2. Performance of the ϵ-greedy and Boltzmann policies for different exploration rates. Other hyperparameters used are: 50,001 timesteps, a learning rate of 0.1

3.3. Back-up: On-policy versus off-policy target

An important aspect that influences the process of tabular reinforcement learning is the way that the state-action value function is updated. There are two ways to execute this back-up. The first is off-policy, where a bootstrapped value is given by the best possible next action, according to the Q-table. This is how the Q-learning algorithm works (see Algorithm 2). The other way is on-policy, where the value is not determined by the best possible action, but by the action actually taken by the agent. This approach is used by the RL algorithm SARSA (see Algorithm 3). After initializing, SARSA takes actions based on the current policy and observes the resulting next state and reward. Then, the Q-value of the current state-action pair is updated, using the action taken by the agent.

3.4. Back-up: Depth of target

When considering back-up, deciding on off-policy versus on-policy is not the only choice that impacts the performance of the agent. So far, when computing the target value, only the next action was taken into consideration. The agent was only looking one step forward, so to speak. But, it is also possible to look further ahead before bootstrapping, summing rewards into a trace with a discount for rewards that lie further in the future. These methods are known as N-step methods, with depth n denoting how many step the agent looks into the future. One of these is N-step Q-learning (see Algorithm 4). It collects a full episode before updating the Q-value of each encountered state-action pair, summing over the discounted rewards.

An extreme case of N-step Q-learning is the Monte Carlo (MC) update, where the update uses the sum of all discounted rewards until the end of the episode, without any bootstrapping. The algorithm for MC is shown in Algorithm 5. Just like in N-step Q-learning, MC first collects a full episode. Then it loops backwards through the encountered state-action pairs, stacking the sum of discounted rewards in reverse and updating the Q-table accordingly.

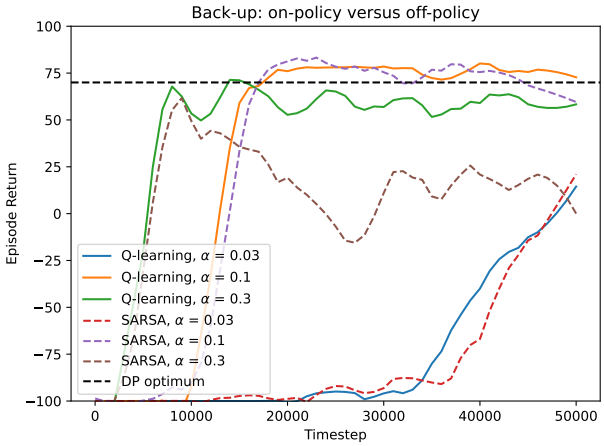

Figure 3. Comparing different approaches of target back-up. Q-learning uses an off-policy back-up with and SARSA uses an on-policy back-up. Both algorithms are tested with learning rates 0.03, 0.1 and 0.3.

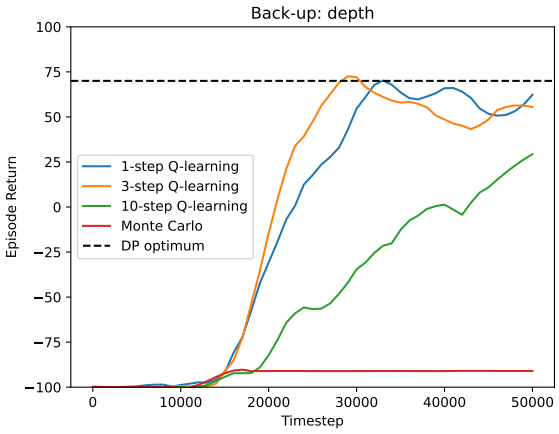

Figure 4. Comparing different back-up depths using depths of 1, 3 and 10 steps and a Monte Carlo update, which considers all steps up to the end of the episode.

4. Discussion

4.1. Dynamic programming

The optimal Q table found using $\gamma = 0.99$ and $\eta = 70$. is visualised in Figure 1. The blue square indicates the initial state and the green square indicates the terminal state. The arrows indicate the greedy action to take in each state. No reward can be received after the agent reaches the goal state: no action is preferred so each action is indicated with an arrow. To ensure that the agent doesn't move away from the goal state, the transition probability is defined such that each action from the goal state will return the goal state: no

movement away from the goal state is permitted. Changing the goal state will yield a different Q table, because different actions will be required to reach this changed goal state (see for example Figure 8).

Using Algorithm 1 to find the optimal policy $\pi^\star$ works well **if** we can afford its high computational cost and we have access to $T(s'|a, s)$. In higher dimensions, the cost of this method will not be worth it (the curse of dimensionality), so other methods need to be applied to deal with this scenario.

4.2. Exploration

If ϵ and τ are chosen well enough, there is not much performance difference between using the ϵ-greedy or Boltzmann policy, but Boltzman is generally very slightly faster in execution time. This is in my opinion because the action selection probability for Boltzman is directly dependent to learned values unlike ϵ-greedy which is always constant. Since I changed both multiple times to get a relatively optimal result, I will note my selected values in the Appendix section. When we turn the reward value for the goal to 0, I believe the convergence will occur with the same speed, though the mean of returns will be different. For large reward values like 100, the curve goes upwards with a large slope, and at some point it becomes descending. That is because the value surpasses the convergence value first because of initial bias in the results. In conclusion, I do not believe changing the rewards would have an effect on the convergence, so long as the goal reward value is bigger than the other steps (which is rationally supposed to be so).

4.3. Back-up: On-policy versus off-policy target

Considering the comparison of Q-learning and SARSA for different learning rates in Figure 3, the off-policy Q-learning approach with a learning rate of 0.1 seems to yield the highest return with the most consistency, implying that the off-policy approach of updating with the best action results in better training due to the direct learning of the optimal policy, whereas SARSA uses more exploration, learning a suboptimal but more realistic policy. The on-policy approach would be useful for a case where exploration is dangerous, and the safest path to reward is the way to go.

A learning rate of 0.1 seems to balance stability and learning speed, as a lower α would converge too slow and a higher α would overshoot the values in the Q-table. Note that for the same learning rates the learning curves of Q-learning and SARSA look similar, indicating that the learning rate dominated the learning process compared to the approach used to update the Q-table.

4.4. Back-up: Depth of target

Figure 4 shows the comparison between different back-up depths for N-step Q-learning, compared to the Monte Carlo approach. The figure shows the highest return for the depths of $n = 1$ and $n = 3$, with the 1-step approach learning the fastest initially, propagating information at every timestep, but the 3-step approach catching up quickly for more timesteps. For $n = 3$, the target value includes more information about future rewards, so it has less bias than for $n = 1$ (which only uses 1 real reward) but it takes longer to see improvements because fewer updates are happening per timestep, leading to the fast increase later. For $n = 10$ or larger, the target update becomes to noisy, with a high variance, considering that the environment is stochastic, leading to a worse performance. But , when given enough time to converge, 10-step Q-learning has the potential to reach the optimal policy, since it uses 10 actual rewards before bootstrapping, resulting in a low bias and the most consistent rising learning curve, in contrast to the learning curves for $n = 1$ and $n = 3$, for which the return decreases again at about 30000 timesteps.

When compared to Figure 3, the 1-step Q-learning seems to learn slower then using Algorithm 2. This could be because regular Q-learning updates every single timestep, while 1-step Q-learning collects a full episode first and then updates.

Coming back to on-policy versus off-policy, the N-step approach lies somewhere in between. The first n rewards are collected using the on-policy approach, resulting in a trace of rewards that reflect the current policy. But, when bootstrapping, the update uses $\max_a \hat{Q}(s_{t+m}, a)$, just like in regular Q-learning.

5. Conclusion

We have covered several methods for finding the optimal path through the created environment. Using DP is a simple method of generating an optimal policy, but requires much computational power and access to the transition probability.

Regarding the Q-learning algorithm, there's no preference between the ϵ-greedy and Boltzman policy, because they perform similarly (with appropriate parameters). In general, Boltzman is slightly faster in execution time. As for the two-goal problem, it mostly outperforms the single-goal problem, but if the second goal is more achievable it performs even remarkably better.

Different back-ups can be used to update the Q-table, with an off-policy back-up generally resulting in a better learning curve. Learning-rate seems to have little influence over the which back-up is more successful. The depth of the target also affects the learning curve, with a smaller N resulting in a better learning curve for N-step Q-learning.

References

Plaat, A. *Deep Reinforcement Learning*. Springer Nature Singapore, 2022. ISBN 9789811906381. doi: 10.1007/978-981-19-0638-1. URL http://dx.doi.org/10.1007/978-981-19-0638-1.

.1. Algorithms

.1.1. Dynamic programming

Below is the pseudocode for the value iteration algorithm used to construct the Q-table in subsection 3.1.

Algorithm 1 Tabular Q-value iteration

Input: n_{states}, n_{actions}, threshold $\eta \in \mathrm{R}_{>0}$
Initialise: A state-action value table Q with shape $(n_{\text{states}}, n_{\text{actions}})$ and $Q(s,a) = 0\ \forall s \in S, a \in A$
Output: Optimal state-action value table $Q^\star$
repeat
  $\Delta \leftarrow 0$
  for each $s \in S$ **do**
    for each $a \in A$ **do**
      $x \leftarrow Q(s,a)$
      $\tilde{Q}(s,a) \leftarrow \sum_{s'}[p(s'|s,a) \cdot (r(s,a,s') + \gamma \cdot \max_{a'} Q(s',a'))]$
      $\Delta \leftarrow \max\left(\Delta, |x - \tilde{Q}(s,a)|\right)$
    end for
  end for
until $\Delta < \eta$

.1.2. Exploration

This is the general pesudo-code implemented for the Q-Learning method as mentioned in subsection 3.2:

Algorithm 2 Tabular Q-learning

Input: exploration parameter, learning rate $\alpha \in (0,1]$, discount factor $\gamma \in [0,1]$, total budget
Initialise: $\hat{Q}(s,a) \leftarrow 0,\ \forall s \in S,\ a \in A$
$s \sim p_0(s)$ /* sample initial state */
while budget not exhausted **do**
  $a \sim \pi(a \mid s)$ /* e.g., ϵ-greedy, softmax */
  $r, s' \sim p(r, s' \mid s, a)$ /* simulate environment */
  $\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha\left[r + \gamma \max_{a'} \hat{Q}(s',a') - \hat{Q}(s,a)\right]$
  if s' is terminal **then**
    $s \sim p_0(s)$ /* reset environment */
  else
    $s \leftarrow s'$
  end if
end while
Return: $\hat{Q}(s,a)$

.1.3. On-policy vs off-policy

The pseudocode for SARSA as described in subsection 3.3:

Algorithm 3 Tabular SARSA

Input: Number of timesteps t_{tot}, Exploration parameter ϵ, learning rate $\alpha \in (0,1]$, discount parameter $\gamma \in [0,1]$
Initialise: $\hat{Q}(s,a) \leftarrow 0\ \forall\ s \in S, a \in A$
$s \sim p_0(s)$
$a \sim \pi(a|s)$
while $t < t_{tot}$ **do**
  $r, s' \sim p(r, s'|a, s)$
  $a \sim \pi(a'|s')$
  $\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \cdot [r + \gamma \hat{Q}(s',a') - \hat{Q}(s,a)]$
  if s' is terminal **then**
    $s \sim p_0(s)$
    $a \sim \pi(a|s)$
  else
    $s \leftarrow s'$
    $a \leftarrow a'$
  end if
end while
Return $\hat{Q}(s,a)$

.1.4. Depth of target

Pseudo-code for N-step Q-Learning and Monte Carlo, as mentioned in subsection 3.2:

Algorithm 4 N-step Q-learning

Input: Number of timesteps t_{tot}, Exploration parameter ϵ, learning rate $\alpha \in (0,1]$, discount parameter $\gamma \in [0,1]$, Maximum episode length T, target depth n
Initialise: $\hat{Q}(s,a) \leftarrow 0\ \forall\ s \in S, a \in A$
while $t < t_{tot}$ **do**
  $s_0 \sim p_0(s)$
  /// Collect Episode
  for $t = 0, \ldots, (T-1)$ **do**
    $a_t \sim \pi(a|s_t)$
    $r_t, s_{t+1} \sim p(r, s'|s_t, a_t)$
    if s_{t+1} is terminal **then**
      break
    end if
  end for
  $T_ep \leftarrow t + 1$
  /// Compute N-step targets and update
  for $t = 0, \ldots, (T_{ep} - 1)$ **do**
    $m = min(n, T_{ep} - t)$
    if s_{t+m} is terminal **then**
      $G_t \leftarrow \sum_{i=0}^{m-1} (\gamma)^i \cdot r_{t+i}$
    else
      $G_t \leftarrow \sum_{i=0}^{m-1} (\gamma)^i \cdot r_{t+i} + (\gamma)^m \cdot \max_a \hat{Q}(s_{t+m}, a)$
    end if
    $\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \cdot [r + \gamma \hat{Q}(s',a') - \hat{Q}(s,a)]$
  end for
end while
Return $\hat{Q}(s,a)$

Algorithm 5 Tabular Monte Carlo RL

Input: Number of timesteps t_{tot}, Exploration parameter ϵ, learning rate $\alpha \in (0, 1]$, discount parameter $\gamma \in [0, 1]$, Maximum episode length T.
Initialise: $\hat{Q}(s, a) \leftarrow 0 \; \forall \; s \in S, a \in A$
while $t < t_{tot}$ **do**
  $s_0 \sim p_0(s)$
  for $t = 0, \ldots, (T-1)$ **do**
    $a_t \sim \pi(a|s_t)$
    $r_t, s_{t+1} \sim p(r, s'|s_t, a_t)$
    if s_{t+1} is terminal **then**
      break
    end if
  end for
  $G_{t+1} \leftarrow 0$
  for $i = t, \ldots, 0$ **do**
    $G_i \leftarrow r_i + \gamma \cdot G_{i+1}$
    $\hat{Q}(s_i, a_i) \leftarrow \alpha * \cdot [G_i - \hat{Q}(s_i, a_i)]$
  end for
end while
Return $\hat{Q}(s, a)$

.2. Supplementary graphs: DP

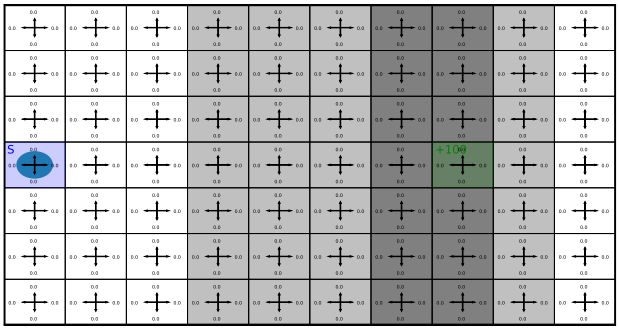

Figure 5. The environment as described in section 3 when initialising the Q table with zeros for all actions a and states s. In this setting the agent will move around randomly until it's reached a maximum number of steps or the goal is reached.

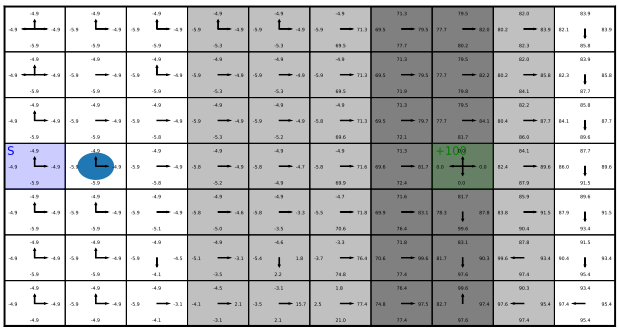

Figure 6. Similar to Figure 5, but now the value iteration algorithm is halfway through its iterations (iteration 6 of the 12). Not every state has an optimal action yet (denoted by an arrow in the direction of the optimal action).

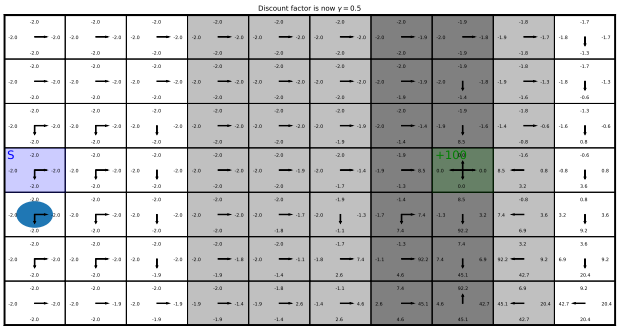

Figure 7. Using $\gamma = 0.5$ and $\eta = 0.99$. You can see that future rewards are taken into account less when building the Q table. At the beginning a path downwards is plotted, even though this will result in the agent going upward due to the stochastic wind.

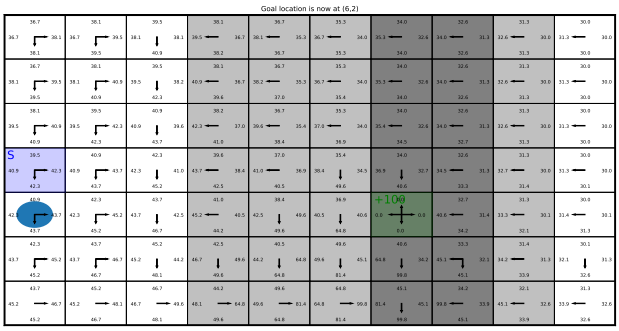

Figure 8. The goal is now at (6,2). Other hyperparameters are set to $\gamma = 0.99$ and $\eta = 1.0$. The agent first moves down and to the right. Approaching the windy area from the left makes more sense, because the agent only has to move through one column of dark grey (strong wind) instead of two when approaching from the right (which as is done in Figure 1).

.3. Exploration:

.3.1. Hyperparameter tuning

As mentioned in the body of the report, the **Experimentation** part, here comes some plots and debates. These are sample snapshots of the running environments: With one goal:

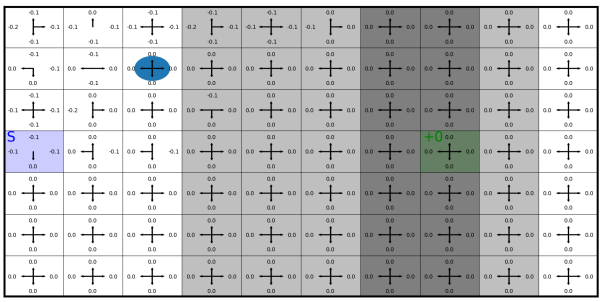

Figure 9. Agent moving with the environment in the 1 goal scenario. Preferable actions are show after every step-out

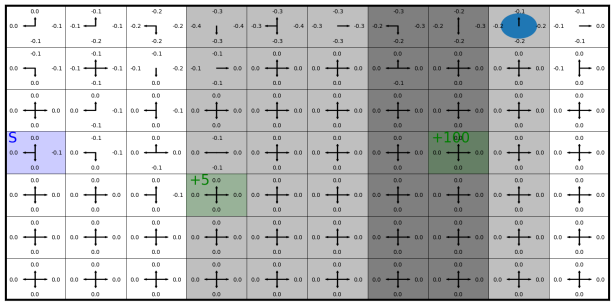

Figure 10. Agent moving with the environment in one of 2 goals scenarios. Preferable actions are show after every step-out

Hyperparameters These are the original values for the following hyperparameters:

```
n_timesteps         = 50001
eval_interval       = 1000
max_episode_length  = 100
gamma               = 1.0
epsilons            = [0.03,0.1,0.3]
learning_rate       = 0.1
temps               = [0.01,0.1,1.0]
```

Resulting Graph:

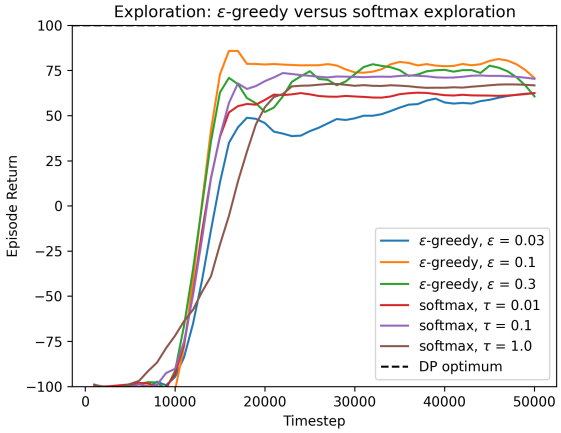

Figure 11. Original hyperparameters graph

The values gradually changed for all learning rate, temperatures, and epsilon values to:

```
n_timesteps         = 15001
eval_interval       = 1000
max_episode_length  = 100
gamma               = 1.0
epsilons            = [1e-5,1e-4,1e-3]
learning_rate       = 0.7
temps               = [1e-4,1e-3,0.01]
```

Result:

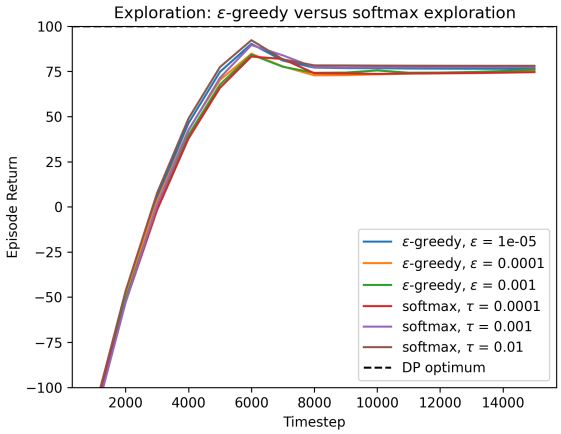

Figure 12. Graph with the new assigned hyperparameters, also demonstrating that with good enough values for both ϵ and τ , they perform similarly

The number of n_timestamps has decreased from 50001 to 15001, because the graph converged much faster. The oscillations and volatile behaviours have also significantly decreased. This also demonstrates that the performance of the two ϵ greedy and softmax boils down to proper choice of the ϵ and τ values. More graphs exist in the Exploration_plots of the assignment repository.

.3.2. EXPLORATION - TWO GOALS:

As for the two goals, I tried the following locations for the second goal: [3,2], [7,4], and [8,4] They are also terminal states, so when the Agent reaches it, the movement stops. This obviously changes the curve so much. We took the positions one at a time (3 experiments) for the second goal and a reward value of 5. The change is that the mean of returns, convergence speed, and hyperparameter effects change. But it also depends so much on where the second goal is, and what its reward value (for mean of returns) also is.
Some facts about two-goals scenario (in comparison with the one-goal scenario):

1. The two-goal scenario generally mostly outperforms the one-goal scenario, because now there are two terminal states with a higher likelihood to be visited.
2. If the second goal is generally much more likely to be visited, then the Returns will orient so much to its reward value. Here are the three respective Graphs with their respective tuned hyperparameters.

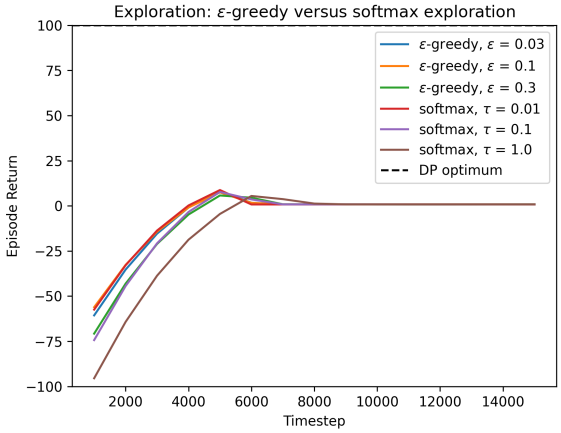

Figure 13. Goal1: [7,3] Goal2: [3,2]

Other parameters: Learning rate: 0.1 # timesteps: 15001

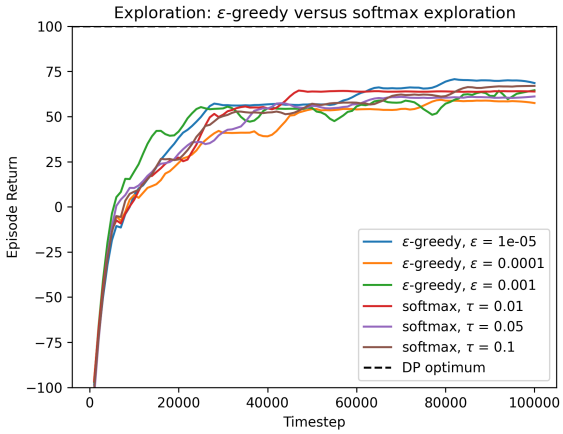

Figure 14. Goal1: [7,3] Goal2: [8,3]

Other parameters: Learning rate: 0.7 # timesteps: 100,001 (slow convergence)

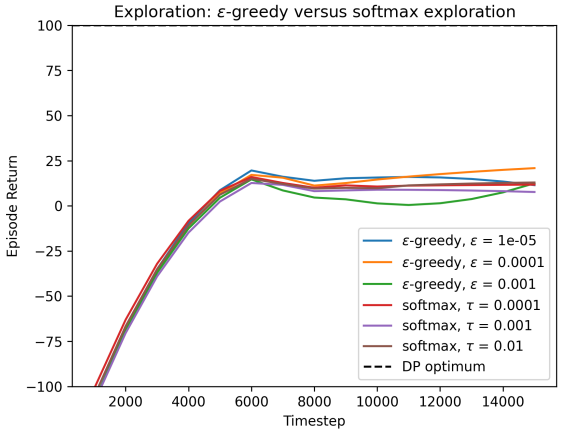

Figure 15. Goal1: [7,3] Goal2: [7,4]

Other parameters: Learning rate: 0.7 # timesteps: 15001

As it complies with the intuition, the Returns of convergence tend to be closer to the goal's Return convergence value **that is more likely to be visited first**. For Goal 1 alone, this value was roughly 75. Now about goal 2 position [3,2], the convergence value is nearly 0 ([3,2] is much more likely to be visited first). About goal 2 position [7,4], the convergence value is around 10 ([7,4] is still a bit more likely to be visited first due to the winds) About goal 2 position [8,3], the convergence value is nearly 60 ([8,4] is a bit less likely to be visited first this time)

The performance also so much depends on the visit likelihood of the second goal. With higher likelihood, the whole scenario converges faster.

Although I said this scenario performs better, the hyperparameters often need to be readjusted for the problem to perform well

Important note about hyperparameters

1. The initial amount of hyperparameters was very conservative, which I believe needs to be so. But as expected, I was mostly able to attain very satisfactory results with less conservative parameters. For example, instead of 20 repetitions, 10 repetitions were also very good. Or 15001 timesteps instead of 50001.
2. Other parameters seriously needed adjustment. Including learning and exploration rates which are the most prominent ones respectively.